

The development model and the open source community

JavaFX is a next-generation client UI toolkit primarily for desktop and embedded devices. It is developed as an open source project, OpenJFX, through contributions from individuals and other companies along the lines of OpenJDK. OpenJFX is free software licensed under the GPL with the class path exception, just like OpenJDK. It is one of the projects beneath the charter of OpenJDK and creates JavaFX libraries as jar files. It is packaged with standard JDK and Java Runtime Environment (JRE), and shipped along with Oracle JDK releases.

JavaFX development is done under the OpenJFX project by individual contributors and companies. Anyone can contribute to JavaFX development; however, the contribution follows a set procedure under which the submitted code is reviewed by the community via open source mailing lists. Review is strictly done on the lines of OpenJDK guidelines and bylaws. Anybody is allowed to download the code, which is a mix of native and Java code, use it in applications or port it to other platforms as well. It is supported on Windows 32/64-bit, Linux 32/64-bit and Mac OS X. The mailing list for OpenJFX development is *openjfx-dev*, which can be subscribed to at the OpenJFX Wiki.

Building and testing

The JavaFX build system uses Mercurial as the source control system and Gradle for building the libraries. Anyone having Windows, Linux or Mac OS X can install Mercurial and extract the source code for JavaFX, the builds of which are fairly complex. These require a set of native compilers and open source tools, for which detailed instructions can be obtained on the OpenJFX Wiki. Post extraction, JavaFX can be locally built and tested. An open source test repository can be used for running regression and the proposed changes. JavaFX product issues can be viewed and filed using the bug database. Anyone willing to contribute patches or enhancements can view open bugs and submit them to the OpenJFX community by following instructions in the Wiki.

Stable JavaFX sources matching JDK8 update releases can be extracted using the *Bu-dev* branch and the Mercurial command, as follows:

```
hg clone http://hg.openjdk.java.net/openjfx/8u-dev/rt
```

The active development stream targeted for JDK9 can be extracted using:

```
hg clone http://hg.openjdk.java.net/openjfx/9-dev/rt
```

The first and simplest task to build JavaFX is *sdk*.

The *sdk* task will compile all Java sources and native sources for your target platform. It will create the appropriate *sdk*



Figure 1: JavaFX architecture

directory, and populate it with the native dynamic libraries and *jfxrt.jar*. JavaFX builds are also supported for cross-builds using Gradle configuration files. Gradle allows you to build JavaFX libraries for platforms which are different from the host platform. It also supports unit testing of the JavaFX build using test tasks. This runs all the unit tests associated with delta changes in the build. Gradle will re-execute only those tests that have changed, or were dependent on code that was changed on subsequent runs. You can, of course, execute Gradle CleanTest to clean all the test results so that they run afresh.

JavaFX architecture

The JavaFX architecture is depicted in Figure 1. In general, a layered architecture puts in UI controls, a set of public JavaFX APIs and the scene graph on top. This public API documentation is available at the Oracle site like all other standard Java API documentation. The starting point for JavaFX application development is a scene graph, shown in purple as part of the API in Figure 1. It is made up of nodes that represent all the visual elements of the application's user interface. It can handle input and can be rendered. This layer comprises a set of public APIs that allow transformation, effects, opacity, etc, to be applied to each node. JavaFX relies more on Web standards, such as CSS, for styling controls.

The Quantum toolkit, in light blue in Figure 1, ties the JavaFX application to the graphics engines Prism and the Glass windowing toolkit. Prism is the JavaFX graphics rendering engine that supports both hardware and software rendering, depending on the hardware beneath. It supports 3D as well. Further, depending on the platform (Windows, Linux or Mac OS X) and the OS versions (Win-XP, Vista, 7, 10) different rendering paths are implemented. The Glass windowing toolkit is the last element in the JavaFX graphics stack and takes care of native operations such as managing the window, timer and surface. The Glass surface is also responsible for managing the event queue and depends on the native system event queue to schedule threads. At any given point, more than one thread is running, which includes the main JavaFX application thread. The media engine supports both audio and video media within the JavaFX application, and supports MP3, AIFF and WAV audio files as well as FLV video files.

Webengine, depicted in brown in Figure 1, is based on